

Using Mach on Windows Outside MozillaBuild

ⓘ Note

These docs still require that you've followed the [Building Firefox On Windows](#) guide.

[MozillaBuild](#) is required to build Firefox on Windows, because it provides necessary unix-y tools such as `sh` and `awk`.

Traditionally, to interact with Mach and the Firefox Build System, Windows developers would have to do so from within the MozillaBuild shell. This could be disadvantageous for two main reasons:

1. The MozillaBuild environment is unix-y and based on `bash`, which may be unfamiliar for developers used to the Windows Command Prompt or Powershell.
2. There have been long-standing stability issues with MozillaBuild - this is due to the fragile interface point between the underlying “MSYS” tools and “native Windows” binaries.

It is now (experimentally!) possible to invoke Mach directly from other command line environments, such as Powershell, Command Prompt, or even a developer-managed MSYS2 environment. Windows Terminal should work as well, for those on the “cutting edge”.

ⓘ Note

If you're using a Cygwin-based environment such as MSYS2, it'll probably be best to use the Windows-native version of Python (as described below) instead of a Python distribution provided by the environment's package manager. Otherwise you'll likely run into compatibility issues:

- Cygwin/MSYS Python will run into compatibility issues with Mach due to its unexpected Unix-y conventions despite Mach assuming it's on a “Windows” platform. Additionally, there may be performance issues.
- MinGW Python will encounter issues building native packages because they'll expect the MSVC toolchain.

⚠ Warning

This is only recommended for more advanced Windows developers: this work is experimental and may run into unexpected failures!

Following are steps for preparing Windows-native (Command Prompt/Powershell) usage of Mach:

1. Install Python

Download Python from the [the official website](#).

⚠ Note

To avoid Mach compatibility issues with recent Python releases, it's recommended to install the 2nd-most recent “major version”. For example, at time of writing, the current modern Python version is 3.14.0, so a safe version to install would be the most recent 3.13 release.

You'll want to download the “Windows installer (64-bit)” associated with the release you've chosen. During installation, ensure that you check the “Add Python 3.x to PATH” option, otherwise you might [encounter issues running Mercurial](#).

⚠ Note

Due to issues with Python DLL import failures with pip-installed binaries, it's not recommended to use the Windows Store release of Python.

2. Modify your PATH

The Python “user site-packages directory” needs to be added to your `PATH` so that packages installed via `pip install --user` (such as `hg`) can be invoked from the command-line.

1. From the Start menu, go to the Control Panel entry for “Edit environment variables for your account”.
2. Double-click the `Path` row in the top list of variables. Click “New” to add a new item to the list.
3. In a Command Prompt window, resolve the Python directory with the command

```
python -c "import site; import os; print(os.path.abspath(os.path.join(site.getusersitepackages(), '..', 'Scripts')))"
```

.
4. Paste the output into the new item entry in the “Edit environment variable” window.
5. Click “New” again, and add the `bin` folder of MozillaBuild: probably `C:\mozilla-build\bin`.
6. Click “OK”.

3. Install Version Control System

If you’re using Mercurial, you’ll need to install it to your Windows-native Python:

```
pip3 install --user mercurial windows-curses
```

If you’re using Git with Cinnabar, follow its [setup instructions](#).

4. Set Powershell Execution Policy

If you’re using Powershell, Windows will raise an error by default when you try to invoke `.\mach.ps1`:

```
.\mach : File <topsrcdir>\mach.ps1 cannot be loaded because running scripts is disabled on this system. For more information, see about_Execution_Policies at https://go.microsoft.com/fwlink/?LinkID=135170. At line:1 char:1
```

To work around this:

1. From the Start menu, type in “Powershell”, then right-click on the best match and click “Run as administrator”
2. Run the command `Set-ExecutionPolicy RemoteSigned`
3. Close the Administrator Powershell window, and open a regular Powershell window
4. Go to your Firefox checkout (likely `C:\mozilla-source\mozilla-unified`)
5. Test the new execution policy by running `.\mach bootstrap` . If it doesn’t immediately fail with the error about “Execution Policies”, then the problem is resolved.

Success

At this point, you should be able to invoke Mach and manage your version control system outside of MozillaBuild.

Tip

[See here](#) for a detailed guide on installing and customizing a development environment with MSYS2, zsh, and Windows Terminal.